

HW 4

Anees Saheba Guddi | Kartheek Alluri | Keon Sadeghi | Manjot Kaur | Rish Jain

Did you find or come across solutions to similar problems by using Generative AI or other sources?: **No**

Real-Time Student Attendance System

Apache Pulsar + Cassandra + Redis (Bloom Filter + HyperLogLog)

Section 1.1 — Environment Setup & Configuration

Services Installed

- Apache Pulsar 3.2.0 — pub/sub messaging broker (Docker, port 6650)
- Redis Stack — in-memory store with Bloom Filter & HyperLogLog modules (Docker, port 6380)
- Apache Cassandra 4.1 — distributed wide-column database (Docker, port 9042)
- Python libraries: pulsar-client, redis, cassandra-driver, faker

Imports

```
import time
import random
import argparse
import threading
from datetime import datetime, timedelta
from collections import defaultdict

try:
    import pulsar
except ImportError:
    print("[ERROR] pulsar-client not installed. Run: pip install pulsar-client")
    pulsar = None

try:
    from faker import Faker
except ImportError:
    print("[ERROR] faker not installed. Run: pip install faker")
    Faker = None

try:
    import redis as redis_lib
except ImportError:
    print("[ERROR] redis not installed. Run: pip install redis")
    redis_lib = None

try:
    from cassandra.cluster import Cluster
    from cassandra.policies import DCAwareRoundRobinPolicy
except ImportError:
    print("[ERROR] cassandra-driver not installed. Run: pip install cassandra-driver")
    Cluster = None
    DCAwareRoundRobinPolicy = None
```

Configuration Code

The script defines all connection parameters at the top-level for easy reference:

```

# PART 1.1 - CONFIGURATION

print("\n" * 70)
print("PART 1.1 - Environment Setup & Configuration")
print("\n" * 70)

# Pulsar
PULSAR_URL = "pulsar://localhost:6650"
PULSAR_TOPIC = "persistent://public/default/attendance"

# Redis
REDIS_HOST = "localhost"
REDIS_PORT = 6380 # redis-stack container (has BF + HLL modules)
REDIS_DB = 0

# Cassandra
CASSANDRA_HOSTS = ["127.0.0.1"]
CASSANDRA_KEYSPACE = "attendance_db"
CASSANDRA_TABLE = "attendance_records"

# Bloom Filter
BF_KEY = "bfinvalid_student_ids" # Redis key for the bloom filter
BF_ERROR_RATE = 0.01 # 1% false-positive rate
BF_CAPACITY = 100_000 # expected number of elements

# HyperLogLog
HLL_KEY_PREFIX = "hll:lecture:" # key pattern: hll:lecture:YYYY-MM-DD-HH

# Student ID ranges
VALID_ID_MIN = 10000
VALID_ID_MAX = 49999 # IDs in this range are "registered" students
INVALID_ID_MIN = 50000
INVALID_ID_MAX = 99999 # IDs in this range are unregistered (simulate bad swipes)

# Simulation parameters
NBR_VALID_STUDENTS = 200 # pre-seed the bloom filter with these many students
PRODUCE_MESSAGES = 30 # total messages the producer will send in demo mode
SWIPE_DELAY_MIN = 0.5 # seconds between swipes (min)
SWIPE_DELAY_MAX = 1.5 # seconds between swipes (max)

print(f"Pulsar : {PULSAR_URL} | Topic: {PULSAR_TOPIC}")
print(f"Redis : {REDIS_HOST}:{REDIS_PORT}")
print(f"Cassandra: {CASSANDRA_HOSTS} | Keyspace: {CASSANDRA_KEYSPACE}")
print(f"Bloom Filter key: {BF_KEY} | error_rate={BF_ERROR_RATE} capacity={BF_CAPACITY}")
print(f"HyperLogLog prefix: {HLL_KEY_PREFIX}")
print()

```

Cassandra Schema

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use `--disable-history` or set 'disabled = true' in the [history] section of cqlshrc.
See <https://cassandra.apache.org/doc/latest/tools/cqlsh.html> for more information.

```

Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.11 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> USE attendance_db;
cqlsh:attendance_db> DESCRIBE TABLE attendance_records;

```

```

CREATE TABLE attendance_db.attendance_records (
    lecture_date date,
    lecture_hour text,
    swipe_time timestamp,
    student_id int,
    PRIMARY KEY ((lecture_date, lecture_hour), swipe_time, student_id)
) WITH CLUSTERING ORDER BY (swipe_time ASC, student_id ASC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_thre
shold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor
'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
cqlsh:attendance_db> exit

```

Output

```
=====
PART 1.1 ~ Running Environment Setup
=====

[Cassandra] Connecting ...
[Cassandra] Creating keyspace 'attendance_db' if not exists ...
[Cassandra] Creating Table 'attendance_records' if not exists ...
[Cassandra] Table 'attendance_records' is ready.

[Redis] Connecting ...
[Redis] Creating Bloom Filter 'bf:valid_student_ids' (error_rate=0.01, capacity=100000) ...
[Redis] Seeding 200 valid student IDs into Bloom Filter ...
[Redis] Bloom Filter seeded with student IDs (sample): [43520, 47616, 46959, 29779, 15072] ...
[Redis] Setup complete.

[Setup] Seeding historical attendance data (5 days * 2 slots) ...
Seeded slot 2026-04-12-09 + 130 attendees
Seeded slot 2026-04-12-14 + 152 attendees
Seeded slot 2026-04-11-09 + 156 attendees
Seeded slot 2026-04-11-14 + 137 attendees
Seeded slot 2026-04-10-09 + 164 attendees
Seeded slot 2026-04-10-14 + 138 attendees
Seeded slot 2026-04-09-09 + 129 attendees
Seeded slot 2026-04-09-14 + 137 attendees
Seeded slot 2026-04-08-09 + 146 attendees
Seeded slot 2026-04-08-14 + 139 attendees
[Setup] Historical seeding done. 1428 records inserted.

[Setup] Environment initialization complete!
Valid student IDs registered : 200
Historical records seeded   : 1428
```

Section 1.2 — Simulating Streamed Student Data (Producer)

Overview

The producer script simulates RFID card readers at a university entrance. It continuously generates student IDs and publishes them as messages to the Pulsar attendance topic. To realistically test the Bloom Filter on the consumer side, messages are a mix of valid registered students and invalid/unknown IDs.

Message Mix Strategy

Probability	ID Range	Description
70%	10000–49999	Registered student (guaranteed to pass Bloom Filter)
15%	10000–49999	Random in valid range — may or may not be registered
15%	50000–99999	Clearly unregistered card (invalid swipe simulation)

Producer Code

```
print("\n" * 70)
print("PART 1.2 ~ Simulating Streamed Student Data (Producer Script)")
print("\n" * 70)

def run_producer(num_messages=None):
    """
    PART 1.2 ~ Continuously generates random student RFID swipes and publishes
    them to the Pulsar 'attendance' topic.

    Each message payload: "Student ID: <id>"
    Mix of valid (registered) and invalid (unregistered) IDs to test
    the Bloom Filter on the consumer side.
    ~ 70% ID drawn from the registered set stored in Redis (guaranteed valid)
    ~ 15% ID drawn from the unregistered set (guaranteed invalid)
    ~ 15% fully random ID (likely invalid ~ simulates bad reads / strangers)
    """
    if pulsar is None or Faker is None:
        print(f"[Producer] Missing dependencies (pulsar-client / faker). Exiting.")
        return

    faker = Faker()

    # Load valid IDs from Redis so we can send realistic registered-student swipes
    r_prod = connect_redis()
    valid_id_list = [int(x) for x in r_prod.smembers("set:valid_student_ids")]
    if not valid_id_list:
        print(f"[Producer] WARNING: No valid IDs found in Redis. Run --mode setup first.")
        valid_id_list = list(range(VALID_ID_MIN, VALID_ID_MAX + 200))
    print(f"[Producer] Loaded {len(valid_id_list)} registered student IDs from Redis.")

    client = pulsar.Client(PULSAR_URL)
    producer = client.create_producer(PULSAR_TOPIC)

    print(f"[*] [Producer] Connected to Pulsar at {PULSAR_URL}")
    print(f"[*] [Producer] Publishing to topic: {PULSAR_TOPIC}")
    print(f"[*] [Producer] Starting RFID swipe simulation ...")

    count = 0
    try:
        while True:
            roll = random.random()
            if roll < 0.70:
                # Registered student swipe (valid - will pass Bloom Filter)
                student_id = random.choice(valid_id_list)
            elif roll < 0.85:
                # Random ID in valid range but NOT necessarily registered
                student_id = faker.random_int(min=VALID_ID_MIN, max=VALID_ID_MAX)
            else:
                # Clearly out-of-range / unknown card (invalid)
                student_id = faker.random_int(min=INVALID_ID_MIN, max=INVALID_ID_MAX)

            message = f"Student ID: {student_id}".encode("utf-8")
            producer.send(message)
            count += 1
            print(f"[Producer] Sent swipe #{count+1} ~ Student ID: {student_id}")

            if num_messages and count == num_messages:
                print(f"[*] [Producer] Sent {count} messages. Stopping.")
                break

            # Simulate realistic delay between student swipes
            delay = random.uniform(SWEEP_DELAY_MIN, SWEEP_DELAY_MAX)
            time.sleep(delay)

    except KeyboardInterrupt:
        print(f"[*] [Producer] Interrupted after {count} messages.")
    finally:
        producer.close()
        client.close()
        print(f"[*] [Producer] Pulsar connection closed.")
```

Output

```
Historical records seeded : 1428

[Demo] Starting Producer in background thread (30 messages) ...
[Producer] Loaded 200 registered student IDs from Redis.
2026-04-13 12:05:53.829 INFO [0x1700eb000] ClientConnection:196 | [<none> -> pulsar://localhost:6650] Create ClientConnection, timeout=10000
2026-04-13 12:05:53.829 INFO [0x1700eb000] ConnectionPool:125 | Created connection for pulsar://localhost:6650-pulsar://localhost:6650-0
2026-04-13 12:05:53.852 INFO [0x171183000] ClientConnection:416 | [[:1]:50731 -> [[:1]:6650]] Connected to broker
2026-04-13 12:05:54.257 INFO [0x171183000] HandlerBase:112 | [persistent://public/default/attendance, ] Getting connection from pool
2026-04-13 12:05:54.318 INFO [0x171183000] BinaryProtoLookupService:86 | Lookup response for persistent://public/default/attendance, lookup-broker-url pulsar://localhost:6650, from [[:1]:50731 -> [[:1]:6650]]
2026-04-13 12:05:54.318 INFO [0x171183000] ProducerImpl:148 | Creating producer for topic:persistent://public/default/attendance, producerName: on [[:1]:50731 -> [[:1]:6650]]
2026-04-13 12:05:54.446 INFO [0x171183000] ProducerImpl:221 | [persistent://public/default/attendance, ] Created producer on broker [[:1]:50731 -> [[:1]:6650]]
2026-04-13 12:05:54.447 INFO [0x171183000] HandlerBase:134 | Finished connecting to broker after 187 ms

[Producer] Connected to Pulsar at pulsar://localhost:6650
[Producer] Publishing to topic: persistent://public/default/attendance
[Producer] Starting RFID swipe simulation ...

[Producer] Sent swipe # 1 - Student ID: 23967

[Demo] Starting Consumer (will process 30 messages) ...
```

Section 1.3 — Processing Attendance Data (Consumer)

Overview

The consumer subscribes to the Pulsar attendance topic and processes each message through a three-stage pipeline: (1) validate the student ID using a Redis Bloom Filter, (2) track unique attendees using a Redis HyperLogLog, and (3) persist the full attendance record to Cassandra.

Processing Pipeline

Step 1	Extract student ID from message payload (format: 'Student ID: 12345')
Step 2	BF.EXISTS on Redis Bloom Filter — reject if ID not in filter (definitely not registered)
Step 3	PFADD to HyperLogLog key hll:lecture:YYYY-MM-DD-HH — tracks unique attendees per lecture slot
Step 4	INSERT into Cassandra attendance_records — persists full record with timestamp

Bloom Filter Implementation

The Bloom Filter is configured with a 1% false-positive rate and capacity of 100,000 elements. All three required Redis commands are used:

HyperLogLog Implementation

For each valid swipe, the student ID is added to a HyperLogLog keyed by lecture date-hour slot. PFADD automatically creates the key if it does not exist:

Screenshot — Code

```

# PART 1.3 – CONSUMER: Processing Attendance Data
print("=" * 70)
print("PART 1.3 – Processing Attendance Data (Consumer/Processor Script)")
print("=" * 70)

INSERT_CQL = """
INSERT INTO {CASSANDRA_TABLE}
    (student_id, lecture_date, lecture_hour, swipe_time)
VALUES (?, ?, ?, ?)
"""

def process_message(msg_data: str, r, cass_session, prepared_insert):
    """
    Process a single attendance message:
    1. Extract student ID
    2. Check Bloom Filter (BF.EXISTS) to validate the ID
    3. If valid → PFADD to HyperLogLog for this lecture slot
    4. Save attendance record to Cassandra
    Returns a dict summarising what happened.
    """
    # Step 1: Extract student ID from "Student ID: 12345"
    try:
        student_id = int(msg_data.strip().split(":")[-1].strip())
    except ValueError:
        return {"status": "parse_error", "raw": msg_data}

    now = datetime.utcnow()
    lecture_date = now.date()
    lecture_hour = now.strftime("%Y-%m-%d-%H") # bucket by hour
    hll_key = f"{HLL_KEY_PREFIX}{lecture_hour}"

    # Step 2: Bloom Filter check – BF.EXISTS <key> <item>
    # Returns 1 if the item may* exist, 0 if it definitely does NOT exist
    bf_result = r.execute_command("BF.EXISTS", BF_KEY, str(student_id))
    is_valid = bool(bf_result)

    result = {
        "student_id": student_id,
        "lecture_date": str(lecture_date),
        "lecture_hour": lecture_hour,
        "timestamp": now.isoformat(),
        "bloom_valid": is_valid,
    }

    if not is_valid:
        print(f" [Consumer] Student {student_id} – REJECTED by Bloom Filter "
              f"(unregistered ID).")
        result["status"] = "rejected_bloom"
        return result

    # Step 3: Valid ID – add to HyperLogLog for this lecture slot
    # PFADD <key> <element> – adds element; auto-creates HLL if key doesn't exist
    pfadd_result = r.pfadd(hll_key, str(student_id))
    # pfadd returns 1 if the HLL was modified (new unique element), 0 otherwise

    print(f" [Consumer] Student {student_id} – ACCEPTED. "
          f"PFADD to '{hll_key}' → {'new unique' if pfadd_result else 'already counted'}.")

    # Step 4: Save to Cassandra
    cass_session.execute(
        prepared_insert,
        (student_id, lecture_date, lecture_hour, now),
    )

    print(f" [Consumer] Saved to Cassandra: "
          f"student={student_id}, date={lecture_date}, hour={lecture_hour}")

    result["status"] = "accepted"
    result["hll_modified"] = bool(pfadd_result)
    return result

def run_consumer(max_messages=None):
    """
    PART 1.3 – Subscribe to the Pulsar 'attendance' topic and process each
    incoming message using Bloom Filter + HyperLogLog (Redis) and Cassandra.
    """
    if pulsar is None or redis_lib is None or Cluster is None:
        print("[Consumer] Missing dependencies. Exiting.")
        return

    print(f"[Consumer] Connecting to Redis at {REDIS_HOST}:{REDIS_PORT} ...")
    r = connect_redis()

    print(f"[Consumer] Connecting to Cassandra at {CASSANDRA_HOSTS} ...")
    cass_session, cluster = connect_cassandra()
    prepared_insert = cass_session.prepare(INSERT_CQL)

    print(f"[Consumer] Connecting to Pulsar at {PULSAR_URL} ...")
    client = pulsar.Client(PULSAR_URL)
    consumer = client.subscribe(
        PULSAR_TOPIC,
        subscription_name="attendance-processor",
        consumer_type=pulsar.ConsumerType.Shared,
    )

    print(f"[Consumer] Subscribed to '{PULSAR_TOPIC}'. Waiting for messages ...")

    count = 0
    try:
        while True:
            msg = consumer.receive(timeout_millis=10_000)
            msg_data = msg.data().decode("utf-8")
            print(f"[Consumer] Received message: '{msg_data}'")

            process_message(msg_data, r, cass_session, prepared_insert)
            consumer.acknowledge(msg)
            count += 1

            if max_messages and count >= max_messages:
                print(f"[Consumer] Processed {count} messages. Stopping.")
                break

    except Exception as exc:
        if "timeout" in str(exc).lower():
            print(f"[Consumer] No more messages (timeout). Processed {count} total.")
        else:
            print(f"[Consumer] Error: {exc}")
    finally:
        consumer.close()
        client.close()
        cluster.shutdown()
        print("[Consumer] Connections closed.")

```

```

def run_consumer(max_messages=None):
    """
    PART 1.3 – Subscribe to the Pulsar 'attendance' topic and process each
    incoming message using Bloom Filter + HyperLogLog (Redis) and Cassandra.
    """
    if pulsar is None or redis_lib is None or Cluster is None:
        print("[Consumer] Missing dependencies. Exiting.")
        return

    print(f"[Consumer] Connecting to Redis at {REDIS_HOST}:{REDIS_PORT} ...")
    r = connect_redis()

    print(f"[Consumer] Connecting to Cassandra at {CASSANDRA_HOSTS} ...")
    cass_session, cluster = connect_cassandra()
    prepared_insert = cass_session.prepare(INSERT_CQL)

    print(f"[Consumer] Connecting to Pulsar at {PULSAR_URL} ...")
    client = pulsar.Client(PULSAR_URL)
    consumer = client.subscribe(
        PULSAR_TOPIC,
        subscription_name="attendance-processor",
        consumer_type=pulsar.ConsumerType.Shared,
    )

    print(f"[Consumer] Subscribed to '{PULSAR_TOPIC}'. Waiting for messages ...")

    count = 0
    try:
        while True:
            msg = consumer.receive(timeout_millis=10_000)
            msg_data = msg.data().decode("utf-8")
            print(f"[Consumer] Received message: '{msg_data}'")

            process_message(msg_data, r, cass_session, prepared_insert)
            consumer.acknowledge(msg)
            count += 1

            if max_messages and count >= max_messages:
                print(f"[Consumer] Processed {count} messages. Stopping.")
                break

    except Exception as exc:
        if "timeout" in str(exc).lower():
            print(f"[Consumer] No more messages (timeout). Processed {count} total.")
        else:
            print(f"[Consumer] Error: {exc}")
    finally:
        consumer.close()
        client.close()
        cluster.shutdown()
        print("[Consumer] Connections closed.")

```

Output

```
[Consumer] Connecting to Redis at localhost:6300 ...
[Consumer] Connecting to Cassandra at ['127.0.0.1'] ...
[Producer] Sent swipe # 2 - Student ID: 35628
[Consumer] Connecting to Pulsar at pulsar://localhost:6650 ...
2026-04-13 12:05:55.989 INFO [0x1f6b8b100] Client:86 | Subscribing on Topic :persistent://public/default/attendance
2026-04-13 12:05:55.990 INFO [0x1f6b8b100] ClientConnection:196 | [none] -> pulsar://localhost:6650] Create ClientConnection, timeout=10000
2026-04-13 12:05:55.990 INFO [0x1f6b8b100] ConnectionPool:126 | Created connection for pulsar://localhost:6650-pulsar://localhost:6650-0
2026-04-13 12:05:55.991 INFO [0x1742bf000] ClientConnection:416 | [::1]:50735 -> [::1]:6650] Connected to broker
2026-04-13 12:05:56.221 INFO [0x1742bf000] HandlerBase:112 | [persistent://public/default/attendance, attendance-processor, 0] Getting connection from pool
2026-04-13 12:05:56.340 INFO [0x1742bf000] BinaryProtoLookupService:86 | Lookup response for persistent://public/default/attendance, lookup-broker-url pulsar://localhost:6650, from [::1]:50735 -> [::1]:6650]
2026-04-13 12:05:56.423 INFO [0x1742bf000] ConsumerImpl:337 | [persistent://public/default/attendance, attendance-processor, 0] Created consumer on broker [::1]:50735 -> [::1]:6650]
2026-04-13 12:05:56.424 INFO [0x1742bf000] HandlerBase:134 | Finished connecting to broker after 282 ms
[Consumer] Subscribed to 'persistent://public/default/attendance'. Waiting for messages ...

[Consumer] Received message: 'Student ID: 65272'
[Consumer] Student 65272 - REJECTED by Bloom Filter (unregistered ID).

[Consumer] Received message: 'Student ID: 41850'
[Consumer] Student 41850 - REJECTED by Bloom Filter (unregistered ID).

[Consumer] Received message: 'Student ID: 12986'
[Consumer] Student 12986 - REJECTED by Bloom Filter (unregistered ID).

[Consumer] Received message: 'Student ID: 23967'
[Consumer] Student 23967 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=23967, date=2026-04-13, hour=2026-04-13-19

[Consumer] Received message: 'Student ID: 35628'
[Consumer] Student 35628 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=35628, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 3 - Student ID: 18624

[Consumer] Received message: 'Student ID: 18624'
[Consumer] Student 18624 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=18624, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 4 - Student ID: 71356

[Consumer] Received message: 'Student ID: 71356'
[Consumer] Student 71356 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 5 - Student ID: 31609

[Consumer] Received message: 'Student ID: 31609'
[Consumer] Student 31609 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 6 - Student ID: 78664

[Consumer] Received message: 'Student ID: 78664'
[Consumer] Student 78664 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 7 - Student ID: 31396

[Consumer] Received message: 'Student ID: 31396'
[Consumer] Student 31396 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=31396, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 8 - Student ID: 23476

[Consumer] Received message: 'Student ID: 23476'
[Consumer] Student 23476 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=23476, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 9 - Student ID: 26058

[Consumer] Received message: 'Student ID: 26058'
[Consumer] Student 26058 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=26058, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 10 - Student ID: 45198

[Consumer] Saved to Cassandra: student=18624, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 4 - Student ID: 71356

[Consumer] Received message: 'Student ID: 71356'
[Consumer] Student 71356 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 5 - Student ID: 31609

[Consumer] Received message: 'Student ID: 31609'
[Consumer] Student 31609 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 6 - Student ID: 78664

[Consumer] Received message: 'Student ID: 78664'
[Consumer] Student 78664 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 7 - Student ID: 31396

[Consumer] Received message: 'Student ID: 31396'
[Consumer] Student 31396 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=31396, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 8 - Student ID: 23476

[Consumer] Received message: 'Student ID: 23476'
[Consumer] Student 23476 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=23476, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 9 - Student ID: 26058

[Consumer] Received message: 'Student ID: 26058'
[Consumer] Student 26058 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=26058, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 10 - Student ID: 45198

[Consumer] Received message: 'Student ID: 45198'
[Consumer] Student 45198 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=45198, date=2026-04-13, hour=2026-04-13-19

[Consumer] Received message: 'Student ID: 31342'
[Producer] Sent swipe # 11 - Student ID: 31342
[Consumer] Student 31342 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=31342, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 12 - Student ID: 13326

[Consumer] Received message: 'Student ID: 13326'
[Consumer] Student 13326 - REJECTED by Bloom Filter (unregistered ID).

[Consumer] Received message: 'Student ID: 31710'
[Producer] Sent swipe # 13 - Student ID: 31710
[Consumer] Student 31710 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=31710, date=2026-04-13, hour=2026-04-13-19
[Producer] Sent swipe # 14 - Student ID: 28893

[Consumer] Received message: 'Student ID: 28893'
[Consumer] Student 28893 - REJECTED by Bloom Filter (unregistered ID).
[Producer] Sent swipe # 15 - Student ID: 16985

[Consumer] Received message: 'Student ID: 16985'
[Consumer] Student 16985 - ACCEPTED. PFADD to 'hll:lecture:2026-04-13-19' + new unique.
[Consumer] Saved to Cassandra: student=16985, date=2026-04-13, hour=2026-04-13-19

[Consumer] No more messages (timeout). Processed 18 total.
2026-04-13 12:06:25.028 INFO [0x1f6b8b100] ConsumerImpl:1361 | [persistent://public/default/attendance, attendance-processor, 0] Closing consumer for topic persistent://public/default/attendance
2026-04-13 12:06:27.127 INFO [0x1742bf000] ConsumerImpl:1345 | [persistent://public/default/attendance, attendance-processor, 0] Closed consumer 0
2026-04-13 12:06:27.144 INFO [0x1f6b8b100] ClientImpl:687 | Closing Pulsar client with 0 producers and 0 consumers
2026-04-13 12:06:27.145 INFO [0x174463000] ClientConnection:1326 | [::1]:50735 -> [::1]:6650] Connection disconnected (refCnt: 1)
2026-04-13 12:06:27.145 INFO [0x174463000] ClientConnection:288 | [::1]:50735 -> [::1]:6650] Destroyed connection to pulsar://localhost:6650-0
[Consumer] Connections closed.
[Producer] Sent swipe # 16 - Student ID: 13364
[Producer] Sent swipe # 17 - Student ID: 47023
```

Section 1.4 — Analyzing Attendance Data

Overview

The analysis module queries both Redis (PFCOUNT for HyperLogLog estimates) and Cassandra (detailed records for 7,538 rows across 11 lecture slots) to generate five attendance insights. Historical data was seeded across 5 days x 2 lecture slots to make insights meaningful.

Screenshot — Code

```
# PART 1.4 — ANALYZING ATTENDANCE DATA
print("--" * 70)
print("PART 1.4 — Analyzing Attendance Data")
print("--" * 70)

def get_hll_unique_count(r, lecture_hour: str) -> int:
    """
    Use Redis PFCOUNT to estimate unique attendees for a given lecture hour.
    PFCOUNT <key> — returns the estimated cardinality (unique elements) of the HLL.
    """
    hll_key = f"{HLL_KEY_PREFIX}:{lecture_hour}"
    count = r.pfcount(hll_key)
    return count

def analyze_attendance():
    """
    PART 1.4 — Generate attendance insights from Cassandra + Redis HyperLogLog.

    Insights generated:
    A. Unique attendee count per lecture slot (via PFCOUNT / HyperLogLog)
    B. Habitual absentees (students below 50% attendance rate)
    C. Habitual latecomers (students who consistently swipe late in the hour)
    D. Most active students (highest total attendance count)
    E. Peak attendance hours (busiest lecture slots)
    """
    if redis_lib is None or Cluster is None:
        print("[Analyze] Missing dependencies. Exiting.")
        return

    print(f"\n[Analyze] Connecting to Redis at {REDIS_HOST}:{REDIS_PORT} ...")
    r = connect_redis()

    print(f"[Analyze] Connecting to Cassandra at {CASSANDRA_HOSTS} ...")
    cass_session, cluster = connect_cassandra()

    # — Fetch all attendance records from Cassandra
    print(f"\n[Analyze] Fetching all records from Cassandra table '{CASSANDRA_TABLE}' ...")
    rows = list(cass_session.execute(
        f"SELECT student_id, lecture_date, lecture_hour, swipe_time "
        f"FROM {CASSANDRA_TABLE}"
    ))
    print(f"[Analyze] Total records fetched: {len(rows)}\n")

    if not rows:
        print("[Analyze] No data found. Run the producer and consumer first.")
        cluster.shutdown()
        return

    # — Build helper data structures
    lectures_per_student = defaultdict(set) # student_id -> {lecture_hour, ...}
    students_per_lecture = defaultdict(set) # lecture_hour -> {student_id, ...}
    swipe_minutes_by_slot = defaultdict(list) # lecture_hour -> [minute_of_swipe, ...]
    late_swipes_by_student = defaultdict(int) # student_id -> count of late swipes
    total_swipes_by_student = defaultdict(int) # student_id -> total swipes

    all_lecture_slots = set()

    for row in rows:
        sid = row.student_id
        lhour = row.lecture_hour
        swipe_dt = row.swipe_time # datetime object from Cassandra
        swipe_minute = swipe_dt.minute if swipe_dt else 0

        lectures_per_student[sid].add(lhour)
        students_per_lecture[lhour].add(sid)
        swipe_minutes_by_slot[lhour].append(swipe_minute)
        total_swipes_by_student[sid] += 1
        all_lecture_slots.add(lhour)

        # Consider a swipe "late" if it happens after minute 10 (10-min grace period)
        if swipe_minute > 10:
            late_swipes_by_student[sid] += 1

    total_lectures = len(all_lecture_slots)

    # INSIGHT A — Unique Attendees Per Lecture (HyperLogLog PFCOUNT)
    print("--" * 70)
    print("INSIGHT A — Estimated Unique Attendees Per Lecture (HyperLogLog)")
    print("  Uses Redis PFCOUNT on HyperLogLog keys built during message processing.")
    print("--" * 70)

    hll_keys = [k for k in r.keys(f"{HLL_KEY_PREFIX}*")]
    if not hll_keys:
        print("  [!] No HyperLogLog keys found. Run the consumer to populate them.")
    else:
        for hll_key in sorted(hll_keys):
            # PFCOUNT returns the estimated number of unique elements
            unique_count = r.pfcount(hll_key)
            lecture_slot = hll_key.replace(HLL_KEY_PREFIX, "")
            actual_count = len(students_per_lecture.get(lecture_slot, set()))
            print(f"  Lecture slot : {lecture_slot}")
            print(f"    HyperLogLog PFCOUNT (estimated unique): {unique_count}")
            print(f"    Cassandra actual unique students      : {actual_count}")
            print()

    # INSIGHT B — Habitual Absentees (attended < 50% of all lectures)

    print("--" * 70)
    print("INSIGHT B — Habitual Absentees (attendance rate < 50%)")
    print("--" * 70)

    threshold = 0.5
    absentees = {}
    for sid, slots in lectures_per_student.items():
        if len(slots) / total_lectures < threshold:
            sid: len(slots)
            for sid, slots in lectures_per_student.items():
                if len(slots) / total_lectures < threshold:
                    rate = attended / total_lectures * 100
                    print(f"  Student {sid:6} — attended {attended}/{total_lectures} "
                          f"lectures ({rate:.1f}%)")

    # INSIGHT C — Habitual Latecomers (late in > 50% of their attended lectures)

    print("\n" + "--" * 70)
    print("INSIGHT C — Habitual Latecomers (late swipe in > 50% of attendances)")
    print("  'Late' = swiped after minute 10 of the lecture hour (10-min grace period)")
    print("--" * 70)

    latecomers = {}
    for sid, late_count in late_swipes_by_student.items():
        attended = total_swipes_by_student[sid]
        if attended > 0 and late_count / attended > 0.5:
            latecomers[sid] = {"late": late_count, "total": attended}

    if not latecomers:
        print("  No habitual latecomers found in current dataset.")
    else:
        print(f"  Students who are late > 50% of the time:\n")
        for sid, info in sorted(
            latecomers.items(), key=lambda x: -x[1]["late"] / x[1]["total"]
        ):
            rate = info["late"] / info["total"] * 100
            print(f"  Student {sid:6} — late {info['late']}/{info['total']} "
                  f"times ({rate:.1f}%)")
```


Output

```
[Consumer] Connections closed.
[Producer] Sent swipe # 16 - Student ID: 13364
[Producer] Sent swipe # 17 - Student ID: 47023
[Producer] Sent swipe # 18 - Student ID: 47024
[Producer] Sent swipe # 19 - Student ID: 68828
[Producer] Sent swipe # 20 - Student ID: 32434
[Producer] Sent swipe # 21 - Student ID: 22229
[Producer] Sent swipe # 22 - Student ID: 39047
[Producer] Sent swipe # 23 - Student ID: 39844
[Producer] Sent swipe # 24 - Student ID: 54778
[Producer] Sent swipe # 25 - Student ID: 44238
[Producer] Sent swipe # 26 - Student ID: 39846
[Producer] Sent swipe # 27 - Student ID: 31651
[Producer] Sent swipe # 28 - Student ID: 38141

[Demo] Running Analysis ...

[Analyze] Connecting to Redis at localhost:6300 ...
[Analyze] Connecting to Cassandra at ['127.0.0.1'] ...

[Analyze] Fetching all records from Cassandra table 'attendance_records' ...
[Producer] Sent swipe # 29 - Student ID: 45689
[Producer] Sent swipe # 30 - Student ID: 54682

[Producer] Sent 30 messages. Stopping.
2026-04-13 12:06:58.312 INFO [0x1709eb000] ProducerImpl:802 | [persistent://public/default/attendance, standalone-0-19] Closing producer for topic persistent://public/default/attendance
2026-04-13 12:06:58.340 INFO [0x171183000] ProducerImpl:766 | [persistent://public/default/attendance, standalone-0-19] Closed producer 0
2026-04-13 12:06:58.342 INFO [0x1709eb000] ClientImpl:687 | Closing Pulsar client with 0 producers and 0 consumers
2026-04-13 12:06:58.343 INFO [0x1742bf000] ClientConnection:1326 | [[:11]:50731 -> [[:11]:6650] Connection disconnected (refCnt: 1)
2026-04-13 12:06:58.343 INFO [0x1742bf000] ClientConnection:288 | [[:11]:50731 -> [[:11]:6650] Destroyed connection to pulsar://localhost:6650-0
[Producer] Pulsar connection closed.
2026-04-13 12:06:58.344 INFO [0x1709eb000] ProducerImpl:759 | Producer - [persistent://public/default/attendance, standalone-0-19] , [batching = off]
[Analyze] Total records fetched: 19160
```

Insight A — Unique Attendees Per Lecture (HyperLogLog PFCOUNT)

Redis PFCOUNT estimates the number of unique students per lecture slot. For the live session (2026-04-13-04), both the HyperLogLog estimate and the Cassandra exact count agree at 162, demonstrating the accuracy of HLL on small-to-medium datasets (within its ~0.81% error bound).

```
Lecture slot : 2026-04-13-04
HyperLogLog PFCOUNT (estimated unique): 162
Cassandra actual unique students      : 162
```

```
Lecture slot : 2026-04-13-18
HyperLogLog PFCOUNT (estimated unique): 58
Cassandra actual unique students      : 58
```

```
Lecture slot : 2026-04-13-19
HyperLogLog PFCOUNT (estimated unique): 46
Cassandra actual unique students      : 46
```

Insight B — Habitual Absentees

Students attending fewer than 50% of the 11 tracked lecture slots are flagged as habitual absentees. The analysis shows a healthy distribution: some students appear in only 1/11 lectures (9.1%) while others attend 5/11 (45.5%), giving a realistic spread for early intervention.

```
Total lectures tracked: 15
Students with < 50% attendance:

Student 20458 - attended 1/15 lectures (6.7%)
Student 32616 - attended 1/15 lectures (6.7%)
Student 21850 - attended 1/15 lectures (6.7%)
Student 16817 - attended 1/15 lectures (6.7%)
Student 39161 - attended 1/15 lectures (6.7%)
Student 28287 - attended 1/15 lectures (6.7%)
Student 37983 - attended 1/15 lectures (6.7%)
Student 40748 - attended 1/15 lectures (6.7%)
Student 48289 - attended 1/15 lectures (6.7%)
Student 15292 - attended 1/15 lectures (6.7%)
Student 22534 - attended 1/15 lectures (6.7%)
Student 19084 - attended 1/15 lectures (6.7%)
Student 26635 - attended 1/15 lectures (6.7%)
Student 48579 - attended 1/15 lectures (6.7%)
Student 28158 - attended 1/15 lectures (6.7%)
Student 37811 - attended 1/15 lectures (6.7%)
Student 38427 - attended 1/15 lectures (6.7%)
Student 22577 - attended 1/15 lectures (6.7%)
Student 12795 - attended 1/15 lectures (6.7%)
Student 21568 - attended 1/15 lectures (6.7%)
Student 39235 - attended 1/15 lectures (6.7%)
Student 21760 - attended 1/15 lectures (6.7%)
Student 36996 - attended 1/15 lectures (6.7%)
Student 44546 - attended 1/15 lectures (6.7%)
Student 15239 - attended 1/15 lectures (6.7%)
Student 24215 - attended 1/15 lectures (6.7%)
Student 38678 - attended 1/15 lectures (6.7%)
Student 44897 - attended 1/15 lectures (6.7%)
Student 24831 - attended 1/15 lectures (6.7%)
Student 14171 - attended 1/15 lectures (6.7%)
Student 15657 - attended 1/15 lectures (6.7%)
Student 30835 - attended 1/15 lectures (6.7%)
Student 28193 - attended 1/15 lectures (6.7%)
Student 18654 - attended 1/15 lectures (6.7%)
Student 26409 - attended 1/15 lectures (6.7%)
Student 46038 - attended 1/15 lectures (6.7%)
Student 32512 - attended 1/15 lectures (6.7%)
Student 14128 - attended 1/15 lectures (6.7%)
Student 29245 - attended 1/15 lectures (6.7%)
Student 35542 - attended 1/15 lectures (6.7%)
Student 23620 - attended 1/15 lectures (6.7%)
Student 13476 - attended 1/15 lectures (6.7%)
Student 39160 - attended 1/15 lectures (6.7%)
Student 28444 - attended 1/15 lectures (6.7%)
Student 36451 - attended 1/15 lectures (6.7%)
Student 39848 - attended 1/15 lectures (6.7%)
Student 27388 - attended 1/15 lectures (6.7%)
Student 45894 - attended 1/15 lectures (6.7%)
Student 46521 - attended 1/15 lectures (6.7%)
Student 29316 - attended 1/15 lectures (6.7%)
Student 22121 - attended 1/15 lectures (6.7%)
Student 30859 - attended 1/15 lectures (6.7%)
Student 32882 - attended 1/15 lectures (6.7%)
Student 44432 - attended 1/15 lectures (6.7%)
Student 35944 - attended 1/15 lectures (6.7%)
Student 42115 - attended 1/15 lectures (6.7%)
Student 38380 - attended 1/15 lectures (6.7%)
Student 21885 - attended 1/15 lectures (6.7%)
Student 18693 - attended 1/15 lectures (6.7%)
Student 13282 - attended 1/15 lectures (6.7%)

Student 49718 - attended 1/15 lectures (6.7%)
Student 27445 - attended 1/15 lectures (6.7%)
Student 17321 - attended 2/15 lectures (13.3%)
Student 19287 - attended 2/15 lectures (13.3%)
Student 23079 - attended 2/15 lectures (13.3%)
Student 15809 - attended 3/15 lectures (20.0%)
Student 12896 - attended 3/15 lectures (20.0%)
Student 23979 - attended 3/15 lectures (20.0%)
Student 46728 - attended 3/15 lectures (20.0%)
Student 19146 - attended 3/15 lectures (20.0%)
Student 21723 - attended 3/15 lectures (20.0%)
Student 22893 - attended 3/15 lectures (20.0%)
Student 36834 - attended 3/15 lectures (20.0%)
Student 21611 - attended 3/15 lectures (20.0%)
Student 22451 - attended 3/15 lectures (20.0%)
Student 46783 - attended 3/15 lectures (20.0%)
Student 25717 - attended 3/15 lectures (20.0%)
Student 45428 - attended 3/15 lectures (20.0%)
Student 35225 - attended 3/15 lectures (20.0%)
Student 12541 - attended 3/15 lectures (20.0%)
Student 45381 - attended 3/15 lectures (20.0%)
Student 44798 - attended 3/15 lectures (20.0%)
Student 19446 - attended 3/15 lectures (20.0%)
Student 44775 - attended 3/15 lectures (20.0%)
Student 22882 - attended 3/15 lectures (20.0%)
Student 13976 - attended 4/15 lectures (26.7%)
Student 41363 - attended 4/15 lectures (26.7%)
Student 11386 - attended 4/15 lectures (26.7%)
Student 30885 - attended 4/15 lectures (26.7%)
Student 42453 - attended 4/15 lectures (26.7%)
Student 11119 - attended 4/15 lectures (26.7%)
Student 15383 - attended 4/15 lectures (26.7%)
Student 45126 - attended 4/15 lectures (26.7%)
Student 16880 - attended 4/15 lectures (26.7%)
Student 32282 - attended 4/15 lectures (26.7%)
Student 26854 - attended 4/15 lectures (26.7%)
Student 13959 - attended 4/15 lectures (26.7%)
Student 27986 - attended 4/15 lectures (26.7%)
Student 19571 - attended 4/15 lectures (26.7%)
Student 27453 - attended 4/15 lectures (26.7%)
Student 26481 - attended 4/15 lectures (26.7%)
Student 22229 - attended 4/15 lectures (26.7%)
Student 48827 - attended 4/15 lectures (26.7%)
Student 12111 - attended 4/15 lectures (26.7%)
Student 37879 - attended 4/15 lectures (26.7%)
Student 47884 - attended 4/15 lectures (26.7%)
Student 41638 - attended 4/15 lectures (26.7%)
Student 32198 - attended 4/15 lectures (26.7%)
Student 27468 - attended 4/15 lectures (26.7%)
Student 12884 - attended 4/15 lectures (26.7%)
Student 48518 - attended 4/15 lectures (26.7%)
Student 36489 - attended 4/15 lectures (26.7%)
Student 25499 - attended 4/15 lectures (26.7%)
Student 38176 - attended 4/15 lectures (26.7%)
Student 36993 - attended 4/15 lectures (26.7%)
Student 22175 - attended 4/15 lectures (26.7%)
Student 11898 - attended 4/15 lectures (26.7%)
Student 14849 - attended 4/15 lectures (26.7%)
Student 21535 - attended 4/15 lectures (26.7%)
Student 16223 - attended 4/15 lectures (26.7%)
Student 49923 - attended 4/15 lectures (26.7%)
Student 26195 - attended 4/15 lectures (26.7%)
Student 39386 - attended 4/15 lectures (26.7%)
Student 47616 - attended 4/15 lectures (26.7%)
Student 36166 - attended 4/15 lectures (26.7%)
Student 34461 - attended 4/15 lectures (26.7%)

Student 46198 - attended 7/15 lectures (46.7%)
Student 23889 - attended 7/15 lectures (46.7%)
Student 32511 - attended 7/15 lectures (46.7%)
Student 16182 - attended 7/15 lectures (46.7%)
Student 39551 - attended 7/15 lectures (46.7%)
Student 25011 - attended 7/15 lectures (46.7%)
Student 23287 - attended 7/15 lectures (46.7%)
Student 46222 - attended 7/15 lectures (46.7%)
Student 16542 - attended 7/15 lectures (46.7%)
Student 31421 - attended 7/15 lectures (46.7%)
Student 43858 - attended 7/15 lectures (46.7%)
Student 25401 - attended 7/15 lectures (46.7%)
Student 25423 - attended 7/15 lectures (46.7%)
Student 32136 - attended 7/15 lectures (46.7%)
Student 17636 - attended 7/15 lectures (46.7%)
Student 43519 - attended 7/15 lectures (46.7%)
Student 19499 - attended 7/15 lectures (46.7%)
Student 43143 - attended 7/15 lectures (46.7%)
Student 37191 - attended 7/15 lectures (46.7%)
Student 18584 - attended 7/15 lectures (46.7%)
Student 38992 - attended 7/15 lectures (46.7%)
Student 17397 - attended 7/15 lectures (46.7%)
Student 47867 - attended 7/15 lectures (46.7%)
Student 11228 - attended 7/15 lectures (46.7%)
Student 26984 - attended 7/15 lectures (46.7%)
Student 11898 - attended 7/15 lectures (46.7%)
Student 37587 - attended 7/15 lectures (46.7%)
Student 48289 - attended 7/15 lectures (46.7%)
Student 46084 - attended 7/15 lectures (46.7%)
Student 37315 - attended 7/15 lectures (46.7%)
Student 38021 - attended 7/15 lectures (46.7%)
Student 24599 - attended 7/15 lectures (46.7%)
Student 18558 - attended 7/15 lectures (46.7%)
Student 36937 - attended 7/15 lectures (46.7%)
Student 44784 - attended 7/15 lectures (46.7%)
Student 38561 - attended 7/15 lectures (46.7%)
Student 33898 - attended 7/15 lectures (46.7%)
Student 38758 - attended 7/15 lectures (46.7%)
Student 43644 - attended 7/15 lectures (46.7%)
Student 23716 - attended 7/15 lectures (46.7%)
Student 37945 - attended 7/15 lectures (46.7%)
Student 38464 - attended 7/15 lectures (46.7%)
Student 25878 - attended 7/15 lectures (46.7%)
Student 17518 - attended 7/15 lectures (46.7%)
Student 26829 - attended 7/15 lectures (46.7%)
Student 33121 - attended 7/15 lectures (46.7%)
Student 24784 - attended 7/15 lectures (46.7%)
Student 18983 - attended 7/15 lectures (46.7%)
Student 48252 - attended 7/15 lectures (46.7%)
Student 48214 - attended 7/15 lectures (46.7%)
Student 32964 - attended 7/15 lectures (46.7%)
Student 43527 - attended 7/15 lectures (46.7%)
Student 22864 - attended 7/15 lectures (46.7%)
Student 18468 - attended 7/15 lectures (46.7%)
Student 38292 - attended 7/15 lectures (46.7%)
Student 42922 - attended 7/15 lectures (46.7%)
Student 39954 - attended 7/15 lectures (46.7%)
Student 16486 - attended 7/15 lectures (46.7%)
Student 17283 - attended 7/15 lectures (46.7%)
Student 41883 - attended 7/15 lectures (46.7%)
Student 12722 - attended 7/15 lectures (46.7%)
Student 47142 - attended 7/15 lectures (46.7%)
Student 11326 - attended 7/15 lectures (46.7%)
Student 32683 - attended 7/15 lectures (46.7%)
Student 18953 - attended 7/15 lectures (46.7%)
```


Students arriving after minute 10 of the lecture hour in more than 50% of their attended sessions are classified as habitual latecomers. The analysis reveals meaningful variation: some students are 100% late while others hover near the 52-55% threshold. 'Late' is defined as arriving after a 10-minute grace period.

Student	11384	-	late	9/9	times	(100.0%)
Student	47496	-	late	5/5	times	(100.0%)
Student	23173	-	late	10/10	times	(100.0%)
Student	13959	-	late	4/4	times	(100.0%)
Student	26481	-	late	4/4	times	(100.0%)
Student	15895	-	late	7/7	times	(100.0%)
Student	45608	-	late	6/6	times	(100.0%)
Student	30887	-	late	5/5	times	(100.0%)
Student	45474	-	late	9/9	times	(100.0%)
Student	33284	-	late	7/7	times	(100.0%)
Student	41295	-	late	5/5	times	(100.0%)
Student	31394	-	late	6/6	times	(100.0%)
Student	39189	-	late	5/5	times	(100.0%)
Student	19146	-	late	3/3	times	(100.0%)
Student	45928	-	late	5/5	times	(100.0%)
Student	25489	-	late	4/4	times	(100.0%)
Student	23677	-	late	6/6	times	(100.0%)
Student	36335	-	late	5/5	times	(100.0%)
Student	48488	-	late	6/6	times	(100.0%)
Student	49569	-	late	7/7	times	(100.0%)
Student	28485	-	late	7/7	times	(100.0%)
Student	48742	-	late	5/5	times	(100.0%)
Student	38931	-	late	6/6	times	(100.0%)
Student	21611	-	late	3/3	times	(100.0%)
Student	46783	-	late	3/3	times	(100.0%)
Student	35111	-	late	6/6	times	(100.0%)
Student	41921	-	late	4/4	times	(100.0%)
Student	28458	-	late	1/1	times	(100.0%)
Student	32616	-	late	1/1	times	(100.0%)
Student	21850	-	late	1/1	times	(100.0%)
Student	16017	-	late	2/2	times	(100.0%)
Student	39161	-	late	2/2	times	(100.0%)
Student	28287	-	late	1/1	times	(100.0%)
Student	37983	-	late	1/1	times	(100.0%)
Student	45768	-	late	1/1	times	(100.0%)
Student	48289	-	late	1/1	times	(100.0%)
Student	15292	-	late	1/1	times	(100.0%)
Student	22534	-	late	1/1	times	(100.0%)
Student	19054	-	late	1/1	times	(100.0%)
Student	26635	-	late	1/1	times	(100.0%)
Student	48579	-	late	1/1	times	(100.0%)
Student	28150	-	late	1/1	times	(100.0%)
Student	37811	-	late	1/1	times	(100.0%)
Student	38427	-	late	1/1	times	(100.0%)
Student	22757	-	late	1/1	times	(100.0%)
Student	12795	-	late	1/1	times	(100.0%)
Student	21568	-	late	1/1	times	(100.0%)
Student	39235	-	late	1/1	times	(100.0%)
Student	21760	-	late	1/1	times	(100.0%)
Student	36996	-	late	1/1	times	(100.0%)
Student	44546	-	late	1/1	times	(100.0%)
Student	15239	-	late	1/1	times	(100.0%)
Student	24215	-	late	1/1	times	(100.0%)
Student	38670	-	late	1/1	times	(100.0%)
Student	44897	-	late	1/1	times	(100.0%)
Student	24831	-	late	1/1	times	(100.0%)

Student	11750	-	late	7/8	times	(87.5%)
Student	28612	-	late	7/8	times	(87.5%)
Student	15574	-	late	7/8	times	(87.5%)
Student	17882	-	late	7/8	times	(87.5%)
Student	19568	-	late	7/8	times	(87.5%)
Student	19568	-	late	7/8	times	(87.5%)
Student	19543	-	late	7/8	times	(87.5%)
Student	44044	-	late	7/8	times	(87.5%)
Student	27366	-	late	7/8	times	(87.5%)
Student	36668	-	late	7/8	times	(87.5%)
Student	17007	-	late	7/8	times	(87.5%)
Student	10128	-	late	6/7	times	(85.7%)
Student	31143	-	late	6/7	times	(85.7%)
Student	19291	-	late	6/7	times	(85.7%)
Student	26441	-	late	6/7	times	(85.7%)
Student	11899	-	late	6/7	times	(85.7%)
Student	31913	-	late	6/7	times	(85.7%)
Student	45757	-	late	6/7	times	(85.7%)
Student	36575	-	late	6/7	times	(85.7%)
Student	25808	-	late	6/7	times	(85.7%)
Student	18899	-	late	6/7	times	(85.7%)
Student	41921	-	late	6/7	times	(85.7%)
Student	21858	-	late	6/7	times	(85.7%)
Student	11664	-	late	6/7	times	(85.7%)
Student	23128	-	late	6/7	times	(85.7%)
Student	35622	-	late	6/7	times	(85.7%)
Student	23098	-	late	6/7	times	(85.7%)
Student	46144	-	late	6/7	times	(85.7%)
Student	41896	-	late	6/7	times	(85.7%)
Student	21265	-	late	6/7	times	(85.7%)
Student	36677	-	late	6/7	times	(85.7%)
Student	33234	-	late	6/7	times	(85.7%)
Student	25192	-	late	6/7	times	(85.7%)
Student	28336	-	late	6/7	times	(85.7%)
Student	42836	-	late	6/7	times	(85.7%)
Student	40659	-	late	6/7	times	(85.7%)
Student	25481	-	late	6/7	times	(85.7%)
Student	25423	-	late	6/7	times	(85.7%)
Student	37191	-	late	6/7	times	(85.7%)
Student	17397	-	late	6/7	times	(85.7%)
Student	47667	-	late	6/7	times	(85.7%)
Student	14552	-	late	6/7	times	(85.7%)
Student	24846	-	late	6/7	times	(85.7%)
Student	24784	-	late	6/7	times	(85.7%)
Student	37237	-	late	6/7	times	(85.7%)
Student	14394	-	late	6/7	times	(85.7%)
Student	37237	-	late	6/7	times	(85.7%)
Student	37587	-	late	6/7	times	(85.7%)
Student	18953	-	late	6/7	times	(85.7%)
Student	48481	-	late	10/12	times	(83.3%)
Student	19125	-	late	5/6	times	(83.3%)
Student	26899	-	late	5/6	times	(83.3%)
Student	74492	-	late	5/6	times	(83.3%)
Student	40887	-	late	5/6	times	(83.3%)
Student	17764	-	late	5/6	times	(83.3%)
Student	36169	-	late	5/6	times	(83.3%)
Student	20225	-	late	5/6	times	(83.3%)
Student	43022	-	late	5/6	times	(83.3%)
Student	32151	-	late	5/6	times	(83.3%)
Student	30870	-	late	5/6	times	(83.3%)
Student	11691	-	late	5/6	times	(83.3%)
Student	32192	-	late	5/6	times	(83.3%)
Student	13529	-	late	5/6	times	(83.3%)
Student	16592	-	late	5/6	times	(83.3%)
Student	44390	-	late	5/6	times	(83.3%)
Student	46112	-	late	5/6	times	(83.3%)

Student	24592	- late	5/9	times	(55.6%)
Student	25779	- late	5/9	times	(55.6%)
Student	23539	- late	5/9	times	(55.6%)
Student	26688	- late	5/9	times	(55.6%)
Student	39147	- late	5/9	times	(55.6%)
Student	22378	- late	5/9	times	(55.6%)
Student	15322	- late	5/9	times	(55.6%)
Student	28421	- late	5/9	times	(55.6%)
Student	29742	- late	5/9	times	(55.6%)
Student	18155	- late	5/9	times	(55.6%)
Student	48555	- late	5/9	times	(55.6%)
Student	18149	- late	5/9	times	(55.6%)
Student	31657	- late	5/9	times	(55.6%)
Student	47267	- late	5/9	times	(55.6%)
Student	14857	- late	5/9	times	(55.6%)
Student	28659	- late	5/9	times	(55.6%)
Student	48176	- late	5/9	times	(55.6%)
Student	17926	- late	5/9	times	(55.6%)
Student	36221	- late	5/9	times	(55.6%)
Student	37531	- late	5/9	times	(55.6%)
Student	4725	- late	5/9	times	(55.6%)
Student	42199	- late	5/9	times	(55.6%)
Student	26956	- late	5/9	times	(55.6%)
Student	12812	- late	5/9	times	(55.6%)
Student	12158	- late	5/9	times	(55.6%)
Student	13925	- late	5/9	times	(55.6%)
Student	48916	- late	5/9	times	(55.6%)
Student	26724	- late	5/9	times	(55.6%)
Student	48233	- late	5/9	times	(55.6%)
Student	38482	- late	5/9	times	(55.6%)
Student	31399	- late	5/9	times	(55.6%)
Student	17862	- late	5/9	times	(55.6%)
Student	15811	- late	5/9	times	(55.6%)
Student	33725	- late	5/9	times	(55.6%)
Student	21580	- late	5/9	times	(55.6%)
Student	45081	- late	5/9	times	(55.6%)
Student	16658	- late	5/9	times	(55.6%)
Student	14321	- late	5/9	times	(55.6%)
Student	29047	- late	5/9	times	(55.6%)
Student	34028	- late	5/9	times	(55.6%)
Student	11423	- late	5/9	times	(55.6%)
Student	45144	- late	5/9	times	(55.6%)
Student	47439	- late	5/9	times	(55.6%)
Student	11343	- late	5/9	times	(55.6%)
Student	28359	- late	5/9	times	(55.6%)
Student	39982	- late	5/9	times	(55.6%)
Student	38582	- late	5/9	times	(55.6%)
Student	37993	- late	5/9	times	(55.6%)
Student	14314	- late	5/9	times	(55.6%)
Student	37723	- late	5/9	times	(55.6%)
Student	44737	- late	5/9	times	(55.6%)
Student	38012	- late	5/9	times	(55.6%)
Student	33590	- late	5/9	times	(55.6%)
Student	18242	- late	5/9	times	(55.6%)
Student	12898	- late	6/11	times	(54.5%)
Student	48558	- late	6/11	times	(54.5%)
Student	29837	- late	6/11	times	(54.5%)
Student	2817	- late	8/15	times	(53.3%)
Student	18817	- late	8/15	times	(53.3%)
Student	11385	- late	8/15	times	(53.3%)
Student	35834	- late	8/15	times	(53.3%)
Student	35581	- late	8/15	times	(53.3%)
Student	15016	- late	8/15	times	(53.3%)
Student	43171	- late	9/17	times	(52.9%)
Student	15043	- late	9/17	times	(52.9%)

Students ranked by total swipe count across all lecture slots. Top performers attend 10 out of 11 tracked sessions and record up to 19 swipes, indicating consistent engagement.

Rank	Student ID	Total Swipes	Lectures Attended
1	32607	19	10
2	32449	19	11
3	35281	18	12
4	26834	18	10
5	46959	18	13
6	20006	18	12
7	10205	18	13
8	47722	18	11
9	17244	17	10
10	43171	17	10

Lecture slots ranked by unique student count. The afternoon slots (14:00) tend to have slightly higher attendance than morning slots, with the busiest session (2026-04-09-14) attracting 776 unique students. This insight helps administrators schedule high-demand courses at optimal times.

Rank	Lecture Slot (YYYY-MM-DD-HH)	Unique Students
1	2026-04-08-09	1897
2	2026-04-11-09	1882
3	2026-04-11-14	1870
4	2026-04-09-14	1849
5	2026-04-09-09	1840
6	2026-04-10-09	1823
7	2026-04-10-14	1810
8	2026-04-08-14	1797
9	2026-04-12-14	1211
10	2026-04-12-09	1185